

Machine learning-enhanced junction modeling for shallow-water channel networks.

Djahou Tognon, Postdoctoral fellowship,
joint work with **Magali Ribot** and **Bruno Galerne**, Université d'Orléans (Orléans, France)

The 20th International Conference on Hyperbolic Problems: Theory, Numerics and Applications (HYP2026), May,
25 - 29, 2026

1. Introduction
2. Hybrid solver
3. Learning setting
4. Numerical results

Introduction

Introduction : 2D shallow water equation on channel network

2D shallow water equation:

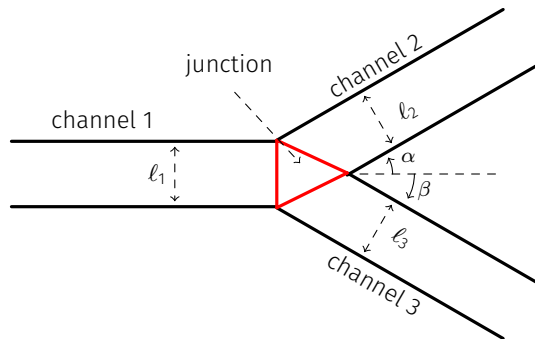
$(x, y) \in \Omega, t \in [0, T],$

$$\partial_t \begin{pmatrix} h \\ hu \\ hv \end{pmatrix} + \partial_x \begin{pmatrix} hu \\ hu^2 + \frac{1}{2}gh^2 \\ huv \end{pmatrix} + \partial_y \begin{pmatrix} hv \\ huv \\ hv^2 + \frac{1}{2}gh^2 \end{pmatrix} = 0,$$

- ▶ h is the water height,
- ▶ u is the x-axis velocity,
- ▶ v is the y-axis velocity,
- ▶ Ω is the channel network.

Channel network: Ω

Channel network parameters: $\alpha, \beta, \ell_j, j = 1, 2, 3.$



Introduction: 1D shallow water on channels + junction conditions

Riemann problem:

$$x \in [a, b], \quad x_0 \in [a, b] \quad t \in [0, T],$$

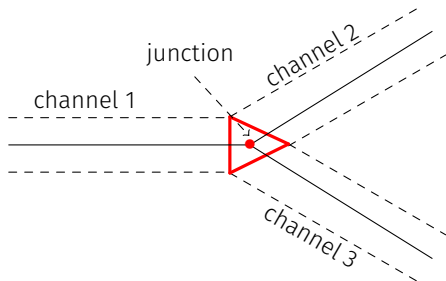
$$\begin{cases} \partial_t h + \partial_x(hu) = 0 \\ \partial_t(hu) + \partial_x(hu^2 + \frac{1}{2}gh^2) = 0, \end{cases} \quad (1D\text{-Eq})$$

$$(h, q)(x, 0) = \begin{cases} (h_l, q_l), & \text{if } x < x_0, \\ (h_r, q_r), & \text{if } x > x_0. \end{cases}$$

where

- ▶ h is the water height and g the gravity,
- ▶ u is the velocity and $q = hu$ the discharge.

1D-Eq on each channel:



$$\begin{cases} \partial_t h_j + \partial_x(h_j u_j) = 0 \\ \partial_t(h_j u_j) + \partial_x(h_j u_j^2 + \frac{1}{2}gh_j^2) = 0, \quad j = 1, 2, 3 \end{cases}$$

coupling with boundary conditions + **junction conditions.**

Junction conditions:

1. Mass conservation:

$$\sum_j h_j v_j = 0, \quad j = 1, 2, 3,$$

2. Equal water height:

$$h_j = h_i, \quad i, j = 1, 2, 3$$

3. Equal energy:

$$h_j + \frac{u_j^2}{2g} = h_i + \frac{u_i^2}{2g}, \quad i, j = 1, 2, 3.$$

Theoretical results:

- ▶ Equal water height: [Colombo, Herty, Sachers, 08] [Marigo, 10],
- ▶ Equal energy: [Goudiaby & Kreiss, 13],
- ▶ Transition from one regime to another (equal water height and mass conservation) [Briani, Piccoli, 18].

Angle dependent conditions:

- ▶ 1D–2D coupling [Bellamoli, Müller, Toro, 2018],
- ▶ Coupling of mass and momentum conservation with Riemann solution [Briani, Puppo, Ribot, 2022].

The aim of our work:

We investigate a neural network model for predicting water dynamics at the junction node, coupled with a finite volume solver on the channels for the full-domain simulation, accounting for different geometries and flow regimes.

Some coupled machine learning approaches with classical numerical solvers:

- ▶ Physics-informed neural network (PINNS) for shallow water [Yongfu, Guofeng, Lida, Jianguo, 2025], [Bilho, Popovych, 2022],
- ▶ Neural Network Surrogates model for shallow water equation [Suresh K., 2023],
- ▶ Finite Volume-Informed Neural Network Framework for 2D Shallow Water Equations [Xiaofeng L., 2026].
- ▶ A neural network closure for the Euler-Poisson system [Bois, Franck, Navoret, Vigon, 2022].

Hybrid solver

Hybrid solver: Finite volume scheme

Discrete notations:

- On channels:

$$\mathbf{h}_j^k = \left(\mathbf{h}_{j,n}^k \right)_{n=0}^{N+1}, \mathbf{u}_j^k = \left(\mathbf{u}_{j,n}^k \right)_{n=0}^{N+1},$$
$$\mathbf{U}_{j,n}^k = (\mathbf{h}_{j,n}^k, \mathbf{u}_{j,n}^k)^T, \quad k = 0, 1, \dots, j = 1, 2, 3$$

- At the junction:

$$\widehat{\mathbf{U}}_j^k = (\widehat{\mathbf{h}}_j^k, \widehat{\mathbf{u}}_j^k)^T, \quad k = 0, 1, \dots, j = 1, 2, 3.$$

Finite volume scheme ($\mathcal{S}$):

- Rewriting (1D-Eq): $\partial_t U + F(U) = 0$,

- On channel j : $n = 1, \dots, N$,

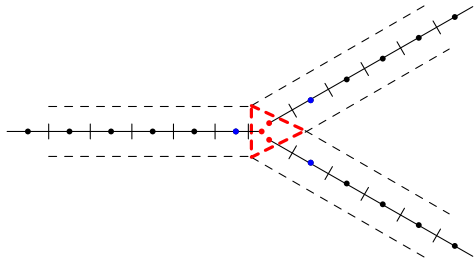
$$\mathbf{U}_{j,n}^{k+1} = \mathbf{U}_{j,n}^k - \frac{\delta t_j}{\delta X_j} \left(F_G(\mathbf{U}_{j,n+1}^k) - F_G(\mathbf{U}_{j,n-1}^k) \right),$$

- F_G computes the flux using F .

- We denote:

$$\begin{pmatrix} \mathbf{U}_{1,N}^{k+1} \\ \delta t_1^{k+1} \end{pmatrix} = \mathcal{S}(\mathbf{U}_{1,N+1}^k),$$

$$\begin{pmatrix} \mathbf{U}_{j,1}^{k+1} \\ \delta t_j^{k+1} \end{pmatrix} = \mathcal{S}(\mathbf{U}_{j,1}^k), \quad j = 2, 3.$$



Hybrid solver: Finite volume scheme coupled with neural network

Hybrid solver : time iteration $k \rightarrow k + 1$

→ Iteration k (available): $(\mathbf{U}_{j,n}^k)_{n=0}^{N+1}$, $j = 1, 2, 3$,
 $\delta t^k = \min\{\delta t_1^k, \delta t_2^k, \delta t_3^k\}$.

► Finite volume solver: $j = 2, 3$,

$$\begin{pmatrix} \mathbf{U}_{1,N}^{k+1} \\ \delta t_1^{k+1} \end{pmatrix} = \mathcal{S}(\mathbf{U}_{1,N+1}^k), \quad \begin{pmatrix} \mathbf{U}_{j,1}^{k+1} \\ \delta t_j^{k+1} \end{pmatrix} = \mathcal{S}(\mathbf{U}_{j,1}^k),$$

► Neural network model $\mathcal{F}(\cdot, \theta)$:

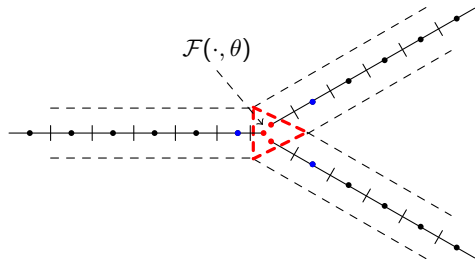
$$\hat{\mathbf{U}}_1^{k+1} = \mathcal{F}\left(\begin{pmatrix} \mathbf{U}_{1,N}^k \\ \delta t^k \end{pmatrix}, \theta\right), \quad \hat{\mathbf{U}}_j^{k+1} = \mathcal{F}\left(\begin{pmatrix} \mathbf{U}_{j,1}^k \\ \delta t^k \end{pmatrix}, \theta\right),$$

$j = 2, 3$.

► Update on boundaries:

$$\mathbf{U}_{1,N+1}^{k+1} = \hat{\mathbf{U}}_1^{k+1}, \quad \mathbf{U}_{j,0}^{k+1} = \hat{\mathbf{U}}_j^{k+1}, \quad j = 2, 3.$$

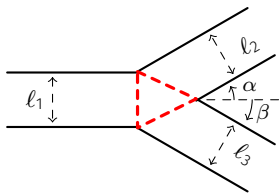
→ Computed: $(\mathbf{U}_{j,n}^{k+1})_{n=0}^{N+1}$, δt_j^{n+1} , $j = 1, 2, 3$.



Learning setting

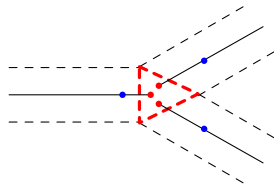
Data for various configurations

- Geometrical parameters: sections ℓ_1, ℓ_2, ℓ_3 , angles α and β .



- Riemann initial conditions.

Input/Output data



For a fixed ℓ_j, α, β and initial condition:

- Input: $X^k = (\mathbf{u}_{1,N}^k, \mathbf{u}_{2,1}^k, \mathbf{u}_{3,1}^k, \delta t^k)$

$$X = (X^k)_{k=1, \dots}$$

- Output: $Y^k = (\hat{\mathbf{u}}_1^k, \hat{\mathbf{u}}_2^k, \hat{\mathbf{u}}_3^k)$

$$Y = (Y^k)_{k=1, \dots}$$

Learning setting: Position of the junction's points

2D simulation:

- Riemann problem:

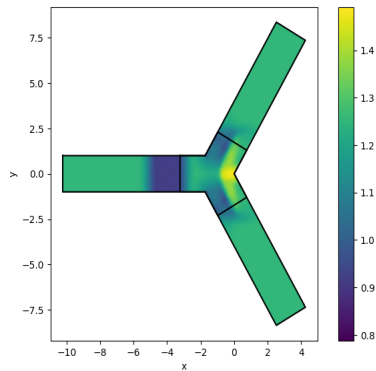
$$(h, q_x, q_y)(x, 0) = \begin{cases} (h_l, q_{x,l}, q_{y,l}) = (1.25, -1.25, 0), & \text{if } x < x_0, \\ (h_r, q_{x,r}, q_{y,r}) = (1.25, 1.25, 0), & \text{if } x > x_0, \end{cases}$$

- $|u_x| = 1 < \sqrt{gh} \approx 3.5$, fluvial regime,
- The Riemann problem generates two rarefaction waves,
- 2D simulation using unstructured mesh,
- $T = 0.3$, $x_0 = -4.0$, and the control-cell area $|K|$ satisfies

$$1.3 \times 10^{-2} \leq \sqrt{|K|} \leq 3.1 \times 10^{-2},$$

- Case of $\ell_1 = \ell_2 = \ell_3 = 2.0$, $\alpha = -\beta = \frac{\pi}{3}$.

Water height:



Learning setting: Position of the junction's points

2D simulation

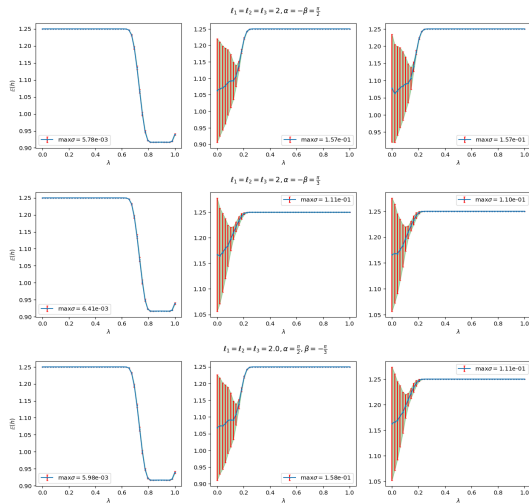
- 2D simulation using unstructured mesh,
- $T = 0.3$, $x_0 = -4.0$, and the control-cell area $|K|$ satisfies

$$1.3 \times 10^{-2} \leq \sqrt{|K|} \leq 3.1 \times 10^{-2}.$$

- 50 sections on each channel parameterized by λ .
- Compute $\mathbb{E}(\mathbf{U}_n^k)$ and $\sigma(\mathbf{U}_n^k)$ over the control cells K_n intersecting each section,
- Junction points position:

$$\sigma(\mathbf{U}_{j,n}^k) \leq \epsilon$$

ϵ , the numerical error.



Learning setting: Position of the junction's points

2D simulation

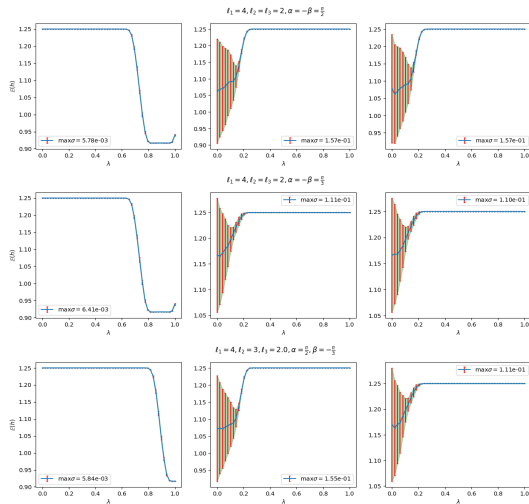
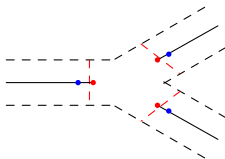
- $T = 0.3$, $x_0 = -4.0$, and the control-cell area $|K|$ satisfies

$$1.3 \times 10^{-2} \leq \sqrt{|K|} \leq 3.1 \times 10^{-2}.$$

- Compute $\mathbb{E}(\mathbf{U}_n^k)$ and $\sigma(\mathbf{U}_n^k)$ over the control cells K_n intersecting each section,
- Junction points position:

$$\sigma(\mathbf{U}_{j,n}^k) \leq \epsilon,$$

ϵ , the numerical error.



Full train and test data:

- ▶ Fixed $|K|$ and $T = 3.0$,
- ▶ M given samples of parameters $\{\ell_1, \ell_2, \ell_3, \alpha, \beta\}$ and a Riemann initial condition,
- ▶ Compute X_m and Y_m , where m denotes the sample label,
- ▶ Define

$$X_{\text{train}} = (X_m)_{m=1, \dots, M}, \quad Y_{\text{train}} = (Y_m)_{m=1, \dots, M},$$

- ▶ Another choice of $\{\ell_1, \ell_2, \ell_3, \alpha, \beta\}$ and a Riemann initial condition for test case

$$X_{\text{test}} = X, \quad Y_{\text{test}} = Y.$$

Test via hybrid solver:

- ▶ Fixed Riemann initial condition used to generate X_{test} and Y_{test} .
- ▶ Parameters θ of $\mathcal{F}$ after training.
- ▶ Hybrid solver simulation up to time small than T to obtain $\mathbf{U}_j$.
- ▶ Error: for $\mathbf{U}_j^e$ the reference solution,

$$\text{Err}_j(\theta, |K|) = \left(\frac{1}{N} \sum_{n=1}^N |\mathbf{u}_{j,n} - \mathbf{u}_{j,n}^e|^2 \right)^{\frac{1}{2}},$$

- ▶ Learning is validated if

$$\text{Err}_j(\theta, |K|) \leq \epsilon.$$

Numerical results

Numerical results: Simple case test ($\ell_1 = 4, \ell_2 = \ell_3 = 2, \alpha = \beta = 0$)

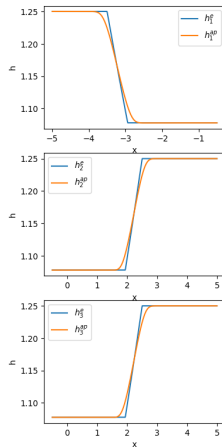
Water height:

Test data:

- Riemann problem:

$$(h, q_x, q_y)(x, 0) = \begin{cases} (h_l, q_{x,l}, q_{y,l}) = (1.25, -0.625, 0), & \text{if } x < x_0, \\ (h_r, q_{x,r}, q_{y,r}) = (1.25, 0.625, 0), & \text{if } x > x_0, \end{cases}$$

- $|u_x| = 0.5 < \sqrt{gh} \approx 3.5$, fluvial regime,
- The Riemann problem generates two rarefaction waves,
- 2D simulation using cartesian mesh,
- $T = 1.0$, $x_0 = -0.5$, and the control-cell area $\sqrt{|K|} = 10^{-2}$.



iter : 4000, 6000, 8000

Neural net. model and loss

- Neural net. model:

$$\mathcal{F}(x, \theta) = x + f(x, \theta),$$

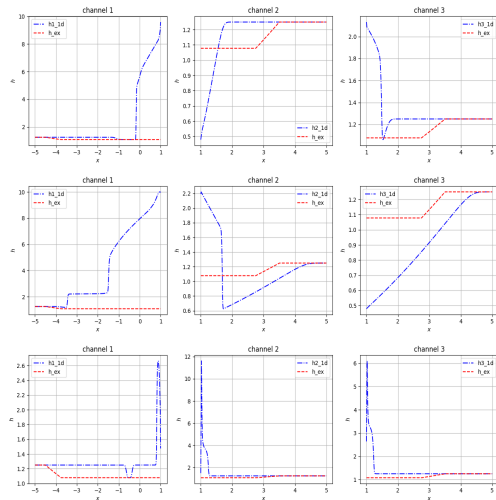
$f(x, \theta)$ is the neural network.

- Statistic loss $\mathcal{L}_1(\theta)$:

$$\mathcal{L}_1(\theta) = \frac{1}{\#X_{train}} \sum_{k=1}^{\#X_{train}} \left| \mathcal{F}(X^k, \theta) - Y^k \right|^2.$$

Issues

Lack of monotonicity from the 1D hybrid solver simulation.



Numerical results: Neural net. model and dynamical learning

iter : 40, 60, 80

Neural net. model and loss

- Neural net. model:

$$\mathcal{F}(x, \theta) = x + f(x, \theta),$$

- Dynamical loss $\mathcal{L}_2(\theta)$: $\mathbf{U}^0 = (\mathbf{U}_j^0)_{j=1,2,3}$ init. condition of (X, Y) :

$$\mathcal{L}_2(\theta) = \frac{1}{\#X-2} \sum_{k=1}^{\#X-2} \left| \mathcal{F}_\theta(S, \mathcal{F}_\theta)(\mathbf{U}_0) - Y^{k+2} \right|^2,$$

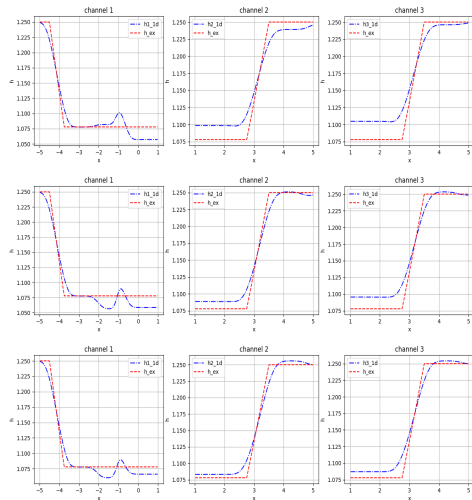
$$\mathcal{F}_\theta(\cdot) = \mathcal{F}(\cdot, \theta),$$

- Loss:

$$\mathcal{L}(\theta) = \omega_1 \mathcal{L}_1(\theta) + \omega_2 \mathcal{L}_2(\theta).$$

Issues

Long time training and gradient propagation issues.



Summary

- ▶ Data generation requires careful design of the junction point positions,
- ▶ Statistical training, as considered here, does not yield good results,
- ▶ The coupling of statistical and dynamical learning can make a significant difference.

Ongoing investigation

- ▶ Investigate the effect of adding monotonicity constraints to the statistical loss,
- ▶ Design new strategies to prevent gradient propagation issues in the dynamical loss,
- ▶ Extend the results to more general geometries and flow regimes,
- ▶ Train using high-order data.

THANK YOU !